SSH Key Authentication on AWS EC2

Practical Lab Report

Experiment: Configure public-key SSH access for a new Linux user

Platform: Amazon EC2 – Amazon Linux 2023
Client: Windows Command Prompt

Student: Saish Chipkar

Objective: Learn how an SSH key pair authenticates a Linux user on a remote EC2 instance.

October 20, 2026

Contents

1 Aim **2**

2 Objectives **2**

3 Requirements **2**

4 Theory and Concepts **2**
4.1 What is SSH? . . . 2
4.2 Public key and private key . . . 2
4.3 How SSH authentication works . . . 3
4.4 Important SSH files . . . 3
4.5 Meaning of the commands used . . . 3
4.6 Linux permission numbers . . . 4

5 Procedure **4**
5.1 Step 1: Launch an EC2 instance and create an AWS key pair . . . 4
5.2 Step 2: Connect to EC2 using the original PEM key . . . 5
5.3 Step 3: Create the new Linux user . . . 6
5.4 Step 4: Generate a separate key pair on Windows . . . 7
5.5 Breakdown of the SSH Key Generation Command . . . 7
5.5.1 Files Generated . . . 7
5.5.2 Working of the Command . . . 8
5.6 Step 5: Prepare the new user's SSH directory . . . 9
5.7 Step 6: Add the public key using `vi` . . . 10
5.8 Step 7: Set File Ownership and Permissions . . . 11
5.8.1 1. Changing File Ownership Using `chown` . . . 11
5.8.2 2. Understanding Linux File Permissions . . . 12
5.8.3 3. Understanding the `chmod 600` Command . . . 12
5.8.4 4. Understanding the `chmod 700` Command . . . 13
5.8.5 5. Applying and Verifying Permissions . . . 13
5.9 Step 8: Exit and test SSH login as the new user . . . 14

6 Results and Observations **15**

7 Troubleshooting Notes **16**

8 Security Best Practices **16**

9 Conclusion **17**

1 Aim

To configure SSH public-key authentication for a newly created Linux user on an Amazon EC2 Linux instance, and to verify that the user can connect from Windows using a locally stored private key.

2 Objectives

- Understand the roles of SSH, a public key, a private key, and a Linux username.
- Generate an Ed25519 SSH key pair on Windows.
- Connect to an EC2 instance using an existing AWS-generated PEM key.
- Create a Linux user and configure that user's SSH authorization file.
- Set appropriate file ownership and permissions.
- Test SSH login as the new Linux user.

3 Requirements

- An AWS account and a running EC2 instance using Amazon Linux 2023.
- The instance's public IPv4 address or public DNS name.
- An existing EC2 private key file, such as `PublicLinux.pem`, that permits initial access.
- Windows Command Prompt or PowerShell with the OpenSSH client installed.
- Administrative access on the EC2 instance to create users and configure their SSH files.

4 Theory and Concepts

4.1 What is SSH?

SSH stands for *Secure Shell*. It is a protocol used to securely access and administer a remote computer over a network. SSH encrypts the connection so that commands and session data are protected in transit.

4.2 Public key and private key

An SSH key pair contains two mathematically related keys:

- **Private key:** Kept secret on the client computer. The SSH client uses it to prove possession of the key. It must not be shared or copied into the server's `authorized_keys` file.
- **Public key:** Safe to distribute. The server stores it in an authorized-keys file for the Linux account that is allowed to authenticate with the matching private key.

The private key is not inherently tied to one Linux user or one EC2 instance. The server's configuration determines which public keys are accepted for each Linux account.

4.3 How SSH authentication works

1. The client requests a connection to a specific host and names the Linux account, for example `saish`.
2. The client offers public-key authentication and proves possession of the matching private key without sending the private key itself.
3. The SSH server checks whether the corresponding public key is authorized for that Linux account.
4. If the key is authorized and network, account, SSH configuration, and permission checks pass, authentication succeeds.

4.4 Important SSH files

Path	Purpose
`~/.ssh/id_ed25519`	Private key generated on the client.
`~/.ssh/id_ed25519.pub`	Matching public key generated on the client.
`/home/saish/.ssh/authorized_keys`	Public keys accepted for the Linux user `saish`.
`/etc/ssh/sshd_config`	SSH server configuration, including authorized-key file settings.

4.5 Meaning of the commands used

`ssh-keygen -t ed25519 -C "saish@ec2"`
Generates an Ed25519 key pair. The `-t` option selects the algorithm; `-C` adds a descriptive comment and does not create a Linux user or grant access.

`ssh -i KEY USER@HOST`
Starts an SSH connection. The `-i` option selects the identity/private-key file, `USER` is the Linux account, and `HOST` is the instance address.

`sudo adduser saish`
Creates the Linux user `saish` with a home directory on Amazon Linux.

`sudo vi authorized_keys`
Opens the file in the `vi` text editor with elevated privileges when required.

`chown saish:saish authorized_keys`
Changes the file owner to user `saish` and the group to `saish`.

`chmod 600 authorized_keys`
Gives the owner read/write permission and gives group and others no permissions.

`chmod 700 /home/saish/.ssh`
Gives the owner read/write/execute permission on the directory and gives group and others no permissions. Execute on a directory allows traversal/access.

`whoami` and `pwd`
`whoami` prints the current Linux username; `pwd` prints the current working directory.

4.6 Linux permission numbers

Linux numeric permissions use read = 4, write = 2, and execute = 1. Therefore, `600` means owner read+write ($4+2=6$), group no access (0), and others no access (0). The public key is not a secret, but strict permissions help prevent unauthorized modification of the trusted-key list.

5 Procedure

5.1 Step 1: Launch an EC2 instance and create an AWS key pair

In the AWS EC2 launch wizard, an instance was configured with the name `PublicServer`. The selected image was Amazon Linux 2023, and the instance type shown was `t3.micro`. A key pair named `PublicLinux` was created with Ed25519 selected and the `.pem` format selected for OpenSSH.

The downloaded PEM file was used as the initial credential to connect to the instance. Keep this key secure. AWS does not provide a way to download the same private key again after its creation.

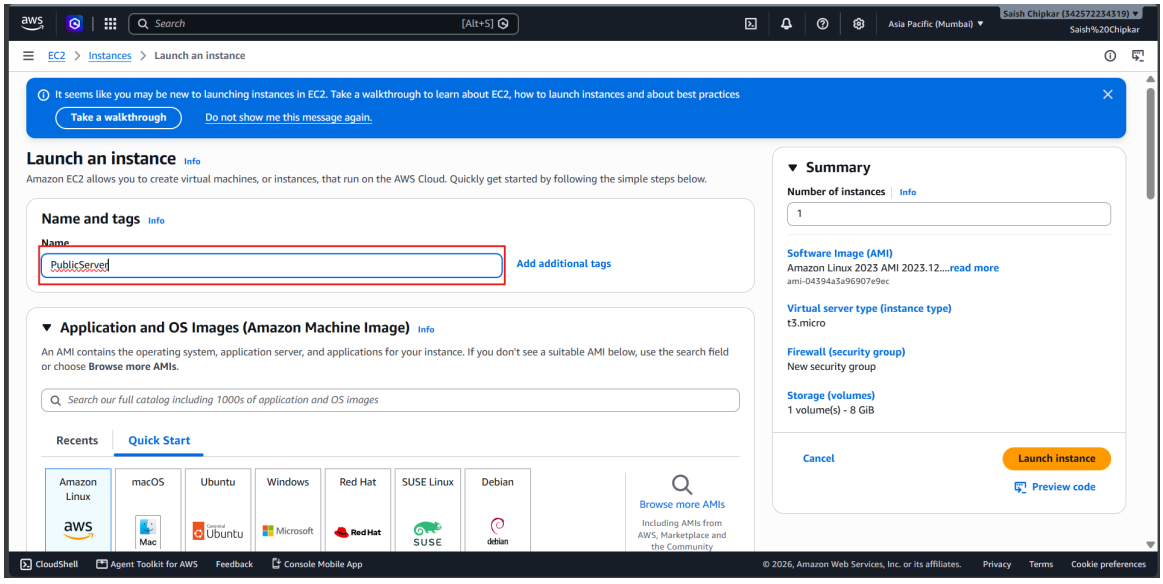


Figure 1: EC2 launch wizard: instance name and selected Amazon Linux image.

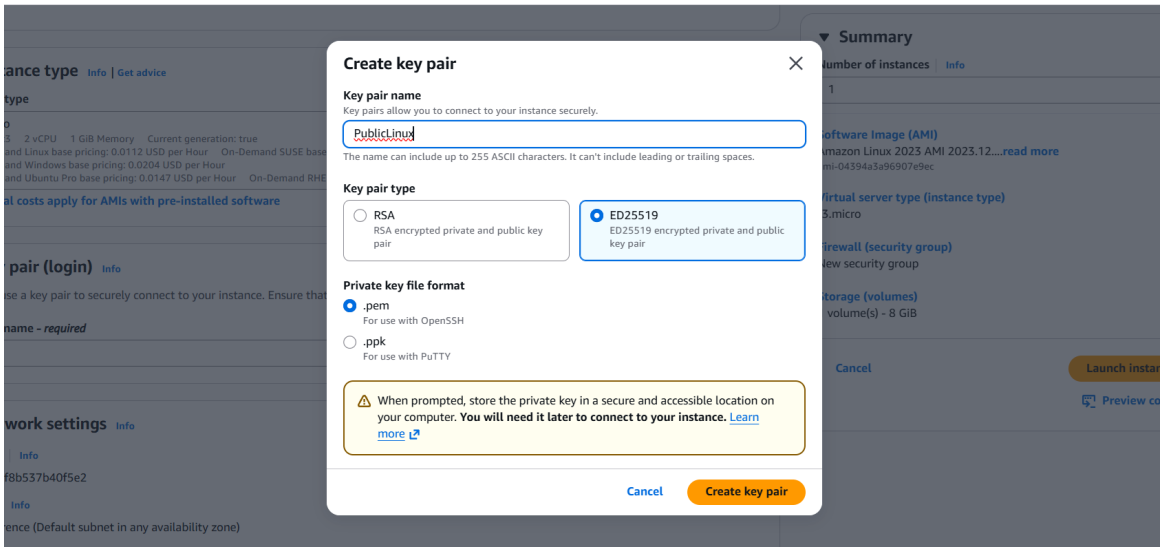


Figure 2: Create-key-pair dialog: key pair name, Ed25519 type, and PEM format.

5.2 Step 2: Connect to EC2 using the original PEM key

From Windows Command Prompt, the initial connection was made using the existing key file. The exact command should use the current instance address and the actual local path to the PEM file.

```
ssh -i "PublicLinux.pem" ec2-user@<EC2-Public-IP>
```

After connecting, `whoami` returned `ec2-user`, and `pwd` returned `/home/ec2-user`. This confirmed the session was running as the default account.

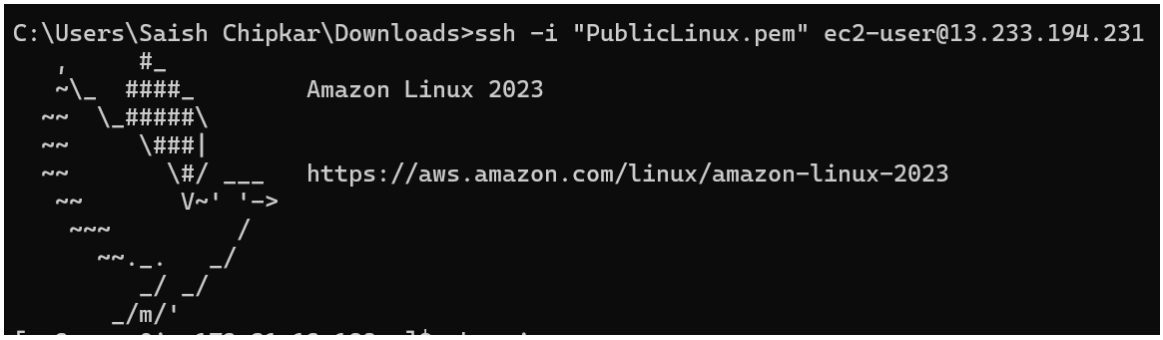


Figure 3: Successful initial connection to Amazon Linux 2023 and verification of the default account.



Figure 4: The public key file is displayed on the server for inspection.

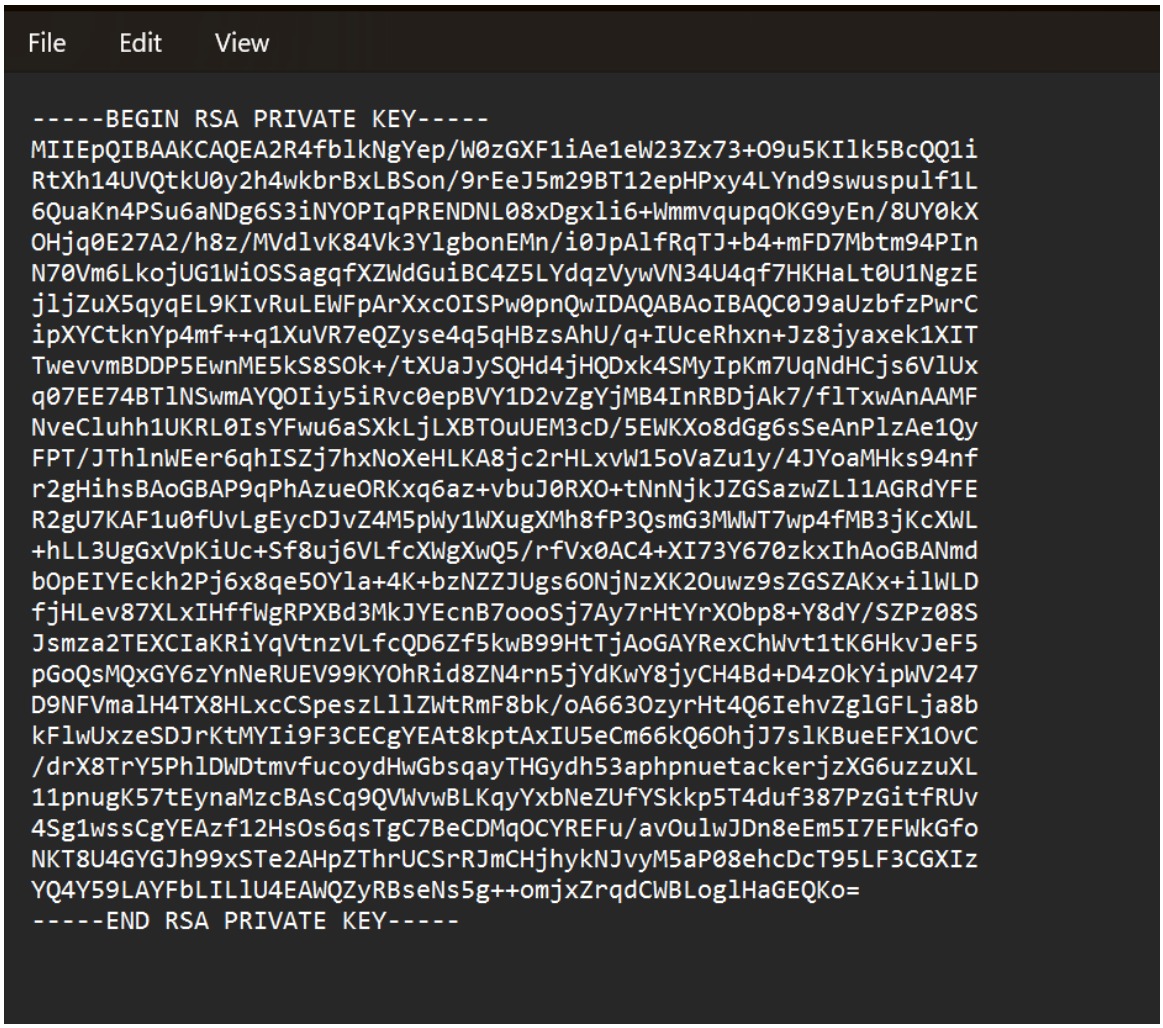


Figure 5: View the private key on our local system.

5.3 Step 3: Create the new Linux user

The following command created the new account:

```
sudo adduser saish
```

The command was run from the existing `ec2-user` session. The user then navigated to the parent directory and listed `/home` to verify that both `ec2-user` and `saish` home directories were present.

```
cd ..
ls
```

```
[ec2-user@ip-172-31-12-198 ~]$ whoami
ec2-user
[ec2-user@ip-172-31-12-198 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-12-198 ~]$ sudo adduser saish
[ec2-user@ip-172-31-12-198 ~]$ cd ..
[ec2-user@ip-172-31-12-198 home]$ ls
ec2-user  saish
```

Figure 6: Create a linux new user.

5.4 Step 4: Generate a separate key pair on Windows

On Windows, a new Ed25519 key pair was generated with a comment identifying its intended use:

```
ssh-keygen -t ed25519 -C "saish@ec2"
```

5.5 Breakdown of the SSH Key Generation Command

The following command is executed in Windows Command Prompt or PowerShell to generate an SSH key pair:

```
ssh-keygen -t ed25519 -C "saish@ec2"
```

Each component of the command has a specific purpose:

- **`ssh-keygen`:** This is the SSH key-generation utility. It generates cryptographic key pairs used for SSH authentication.
- **`-t`:** This option specifies the type of cryptographic key to generate.
- **`ed25519`:** This specifies the Ed25519 algorithm, which uses elliptic-curve cryptography to generate public and private keys.
- **`-C`:** This option adds a comment to help identify the generated key.
- **`"saish@ec2"`:** This is the comment attached to the public key. It is only an identification label and does not create a Linux user or grant access to the EC2 instance.

5.5.1 Files Generated

After executing the command and accepting the default file location, two files are generated in the Windows user's SSH directory:

- **`id_ed25519`:** The private key. It must be kept secret on the client computer and used by the SSH client to prove possession of the key.
- **`id_ed25519.pub`:** The public key. It can be copied to the EC2 instance and added to the intended Linux user's `.ssh/authorized_keys` file.

5.5.2 Working of the Command

1. The `ssh-keygen` utility starts the key-generation process.
2. The `-t ed25519` option selects the Ed25519 algorithm.
3. The `-C "saish@ec2"` option adds a descriptive comment to identify the key.
4. The utility generates a mathematically related public key and private key.
5. The user specifies the file location and may set a passphrase to protect the private key.
6. The generated public key can then be installed in the appropriate Linux user's `authorized_keys` file on the EC2 instance.

Important: Generating the key pair does not automatically authorize SSH access. The corresponding public key must be authorized on the server, and the client must use the matching private key to authenticate.

The default path was accepted:

```
C:\Users\<WindowsUser>\.ssh\id_ed25519
C:\Users\<WindowsUser>\.ssh\id_ed25519.pub
```

The first file is the private key and the `.pub` file is the public key. A passphrase can be set to add protection to the private key at rest. The command prompt showed that the files were saved and displayed a fingerprint for identification.

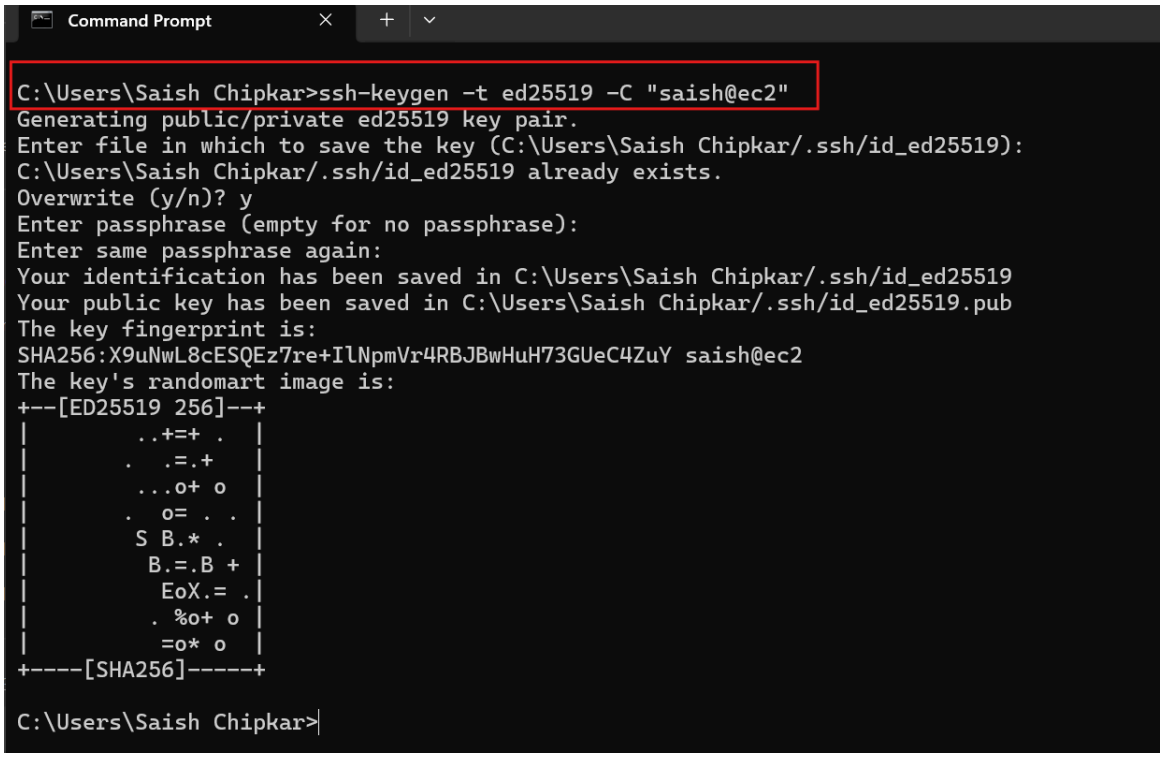


Figure 7: Windows command prompt showing key-pair generation and the save location.

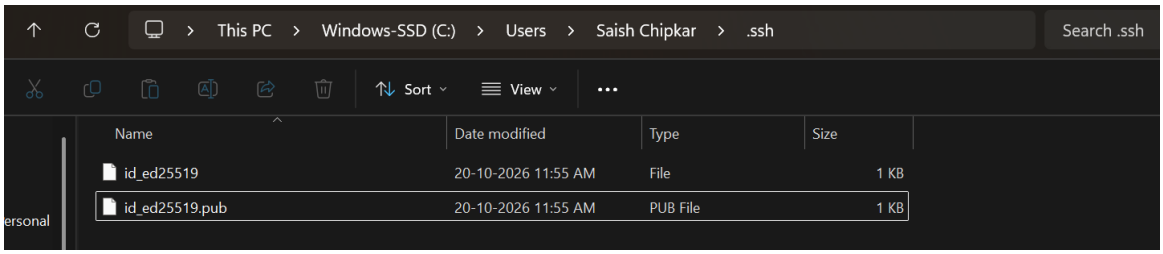


Figure 8: The generated private-key file and public-key file in the Windows `.ssh` directory.

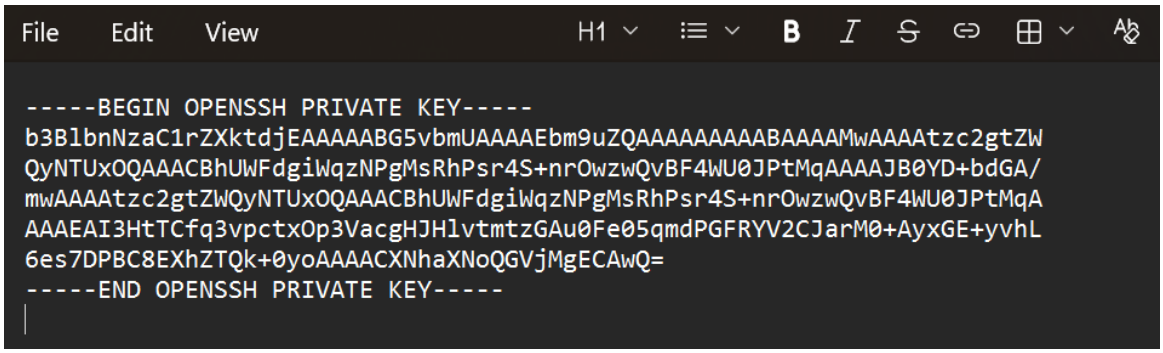


Figure 9: The private key generated

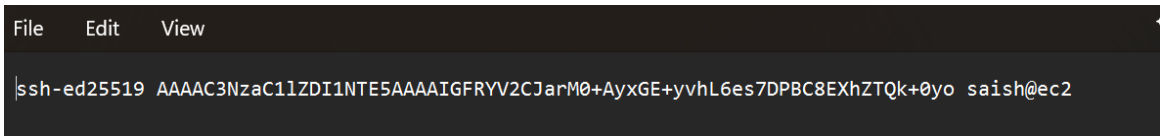


Figure 10: The public key in OpenSSH one-line format. Only the public key should be copied to the server.

5.6 Step 5: Prepare the new user's SSH directory

The user switched to a root shell for administrative configuration and navigated to `/home/saish`. The directory listing showed the standard home-directory files. Then the SSH directory was created.

```
sudo su
cd /home/saish
ls -a
mkdir .ssh
cd .ssh
```

The `.ssh` directory is hidden because its name starts with a dot. The usual location for a user's SSH authorization file is `/home/saish/.ssh/authorized_keys`. Although paths can be customized in SSH server configuration, this is the conventional default.

```
[ec2-user@ip-172-31-12-198 ~]$ sudo su
[root@ip-172-31-12-198 ec2-user]# cd ..
[root@ip-172-31-12-198 home]# cd saish
[root@ip-172-31-12-198 saish]# whoami
root
[root@ip-172-31-12-198 saish]# pwd
/home/saish
[root@ip-172-31-12-198 saish]# ls -a
.  ..  .bash_logout  .bash_profile  .bashrc
[root@ip-172-31-12-198 saish]# mkdir .ssh
[root@ip-172-31-12-198 saish]# cd .ssh
[root@ip-172-31-12-198 .ssh]# sudo vi authorized_keys
```

Figure 11: Root shell navigating to the newly created user's home directory and creating `.ssh`.

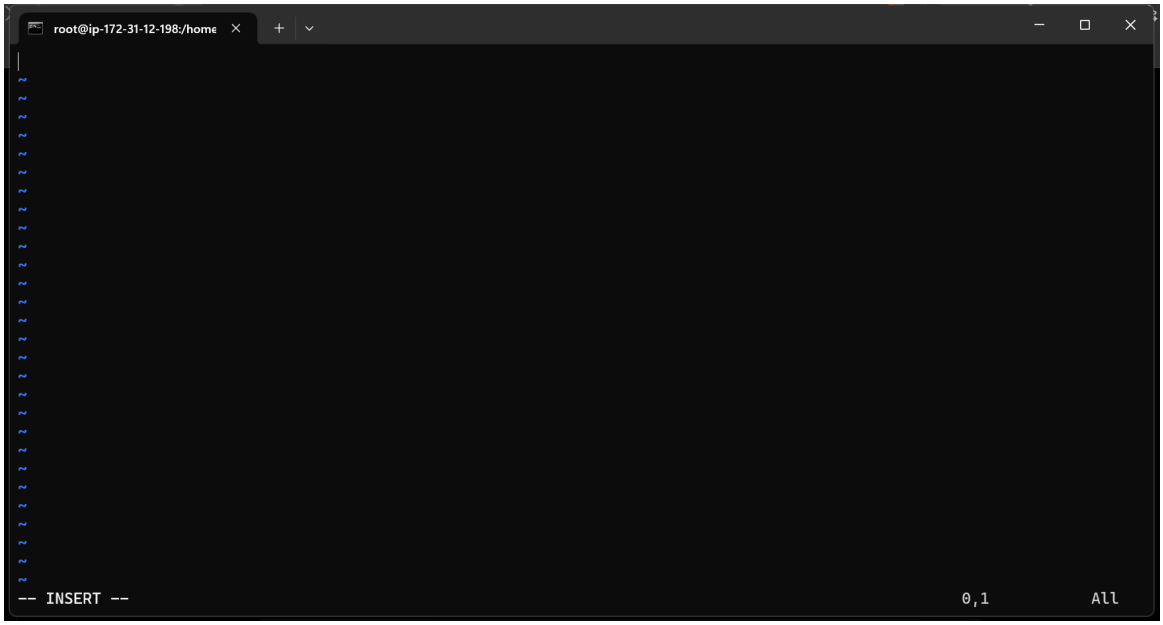


Figure 12: The `vi` editor opened for the `authorized_keys` file.

5.7 Step 6: Add the public key using `vi`

The public key from `id_ed25519.pub` was copied from Windows and pasted into the file. The public key should be a single line beginning with `ssh-ed25519`, followed by the key data and the optional comment.

```
vi authorized_keys
```

In `vi`, press `i` to enter Insert mode, paste the public key, press `Esc`, and enter `:wq` to save and quit. Only the public key belongs in this file; never paste the private key here.

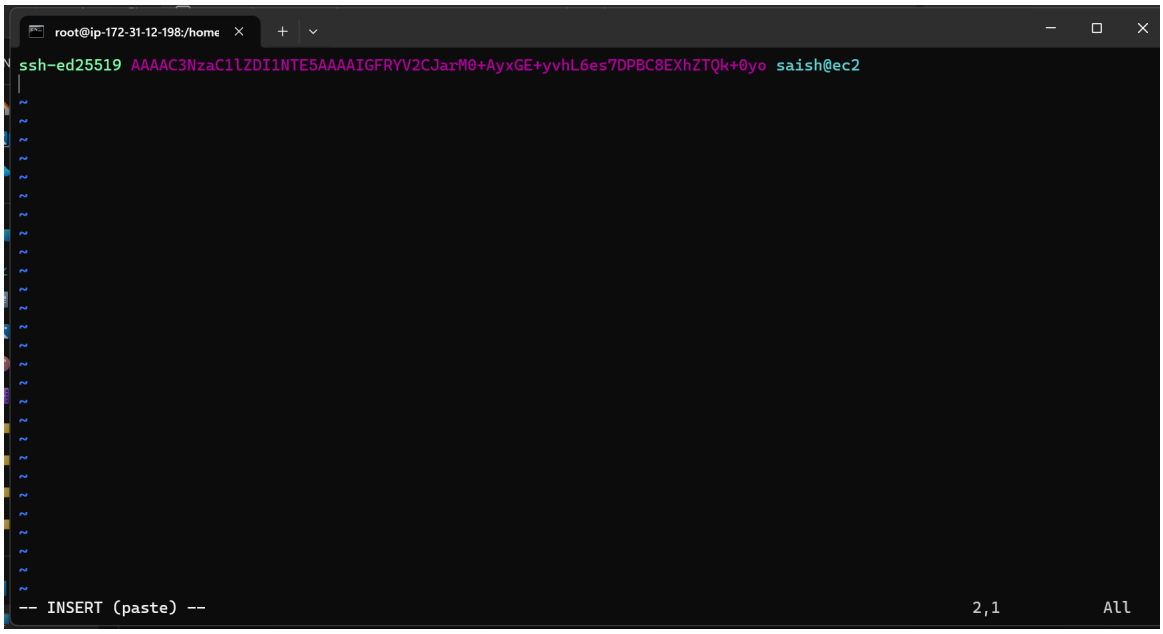


Figure 13: The public key pasted into `authorized_keys` using `vi`.

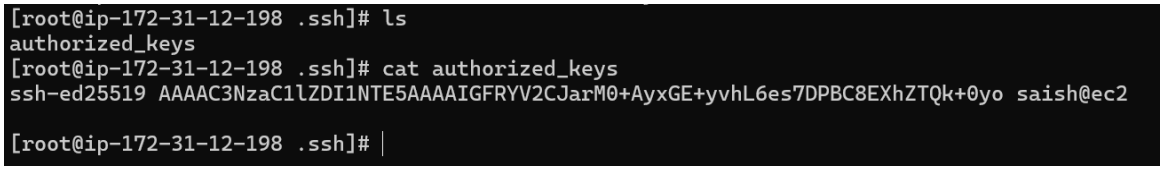


Figure 14: Verification that `authorized_keys` contains the public key in OpenSSH format.

5.8 Step 7: Set File Ownership and Permissions

The file `authorized_keys` was initially created while using the root account. Therefore, its ownership was changed to the intended Linux user, `saish`. Appropriate permissions were then configured to protect the SSH authentication process.

5.8.1 1. Changing File Ownership Using `chown`

```
chown saish:saish authorized_keys
```

The `chown` command changes the ownership of a file or directory.

- `saish` before the colon specifies the new owner of the file.
- `saish` after the colon specifies the new group associated with the file.
- `authorized_keys` is the file whose ownership is being changed.

After executing this command, the file is owned by the Linux user `saish` and belongs to the group `saish`.

5.8.2 2. Understanding Linux File Permissions

Linux permissions control which users can read, modify, or execute files and access directories. Permissions are divided into three categories:

- **Owner (u):** The user who owns the file.
- **Group (g):** Users belonging to the file's associated group.
- **Others (o):** All other users.

Linux uses three basic permission types:

Symbol	Permission	Numeric Value
r	Read	4
w	Write	2
x	Execute	1
-	No permission	0

The numeric permission of each category is calculated by adding the values of the permissions granted.

For example, read and write permissions have a combined value of $4 + 2 = 6$. Read, write, and execute permissions have a combined value of $4 + 2 + 1 = 7$.

5.8.3 3. Understanding the `chmod 600` Command

```
chmod 600 authorized_keys
```

The `chmod` command changes file permissions. In the numeric value `600`, each digit represents the permissions assigned to the owner, group, and others.

Category	Digit	Permission
Owner	6	Read (4) + Write (2)
Group	0	No permissions
Others	0	No permissions

Therefore, `600` means that only the file owner has read and write permissions. Group members and other users have no access through the file's standard permission bits.

The resulting permission string is:

```
-rw-------
```

The first hyphen indicates that this is a regular file. The remaining nine characters represent permissions for the owner, group, and others.

- `rw-`: The owner can read and modify the file but cannot execute it.
- `--`: The group has no permissions.
- `--`: Others have no permissions.

The `authorized_keys` file contains public keys that determine which keys are trusted for the account. Although public keys are not secret, restricting write access prevents unauthorized users from adding or changing trusted keys.

5.8.4 4. Understanding the `chmod 700` Command

```
chmod 700 /home/saish/.ssh
```

The `.ssh` directory stores SSH-related files, including the `authorized_keys` file.
The permission value `700` is interpreted as follows:

Category	Digit	Permission
Owner	7	Read (4) + Write (2) + Execute (1)
Group	0	No permissions
Others	0	No permissions

The resulting directory permission string is:

```
drwx------
```

The initial `d` indicates that the object is a directory.
For directories:

- **Read (r):** Allows listing directory entries, subject to other access checks.
- **Write (w):** Allows creating, deleting, or renaming entries, subject to directory permissions.
- **Execute (x):** Allows traversing the directory and accessing entries when other required permissions are satisfied.

Thus, `700` allows the owner to access and manage the `.ssh` directory, while group members and others receive no permissions through these permission bits.

5.8.5 5. Applying and Verifying Permissions

The following commands ensure that the SSH directory and its authorization file have the intended ownership and permissions:

```
chown saish:saish /home/saish/.ssh
chmod 700 /home/saish/.ssh
chown saish:saish /home/saish/.ssh/authorized_keys
chmod 600 /home/saish/.ssh/authorized_keys
```

These commands should be executed by root or an authorized administrator. If using a regular account, prefix the commands with `sudo` where required.

The configuration can be verified using:

```
ls -ld /home/saish/.ssh
ls -l /home/saish/.ssh/authorized_keys
```

The expected permission strings are similar to:

```
drwx------  ... saish saish ... /home/saish/.ssh
-rw-------  ... saish saish ... authorized_keys
```

The exact file sizes, timestamps, and other displayed details may differ.

Important: The parent directory `/home/saish` must also permit the user to reach the SSH directory. SSH checks ownership and permissions when deciding whether to trust the authorization file. Incorrect permissions or ownership can cause public-key authentication to fail.

```
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-r--r--. 1 root root 92 Oct 20 06:35 authorized_keys
[root@ip-172-31-12-198 .ssh]# chown saish:saish authorized_key
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-r--r--. 1 saish saish 92 Oct 20 06:35 authorized_keys
```

Figure 15: The ownership of `authorized_keys` was changed from root to `saish`.

```
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-r--r--. 1 saish saish 92 Oct 20 06:35 authorized_keys
[root@ip-172-31-12-198 .ssh]# chmod 600 authorized_keys
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-------. 1 saish saish 92 Oct 20 06:35 authorized_keys
[root@ip-172-31-12-198 .ssh]#
```

Figure 16: The file permissions were changed to `600`, allowing only the owner to read and write the file.

```
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-r--r--. 1 root root 92 Oct 20 06:35 authorized_keys
[root@ip-172-31-12-198 .ssh]# chown saish:saish authorized_key
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-r--r--. 1 saish saish 92 Oct 20 06:35 authorized_keys
```

Figure 17: The ownership of `authorized_keys` was changed from root to `saish`.

```
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-r--r--. 1 saish saish 92 Oct 20 06:35 authorized_keys
[root@ip-172-31-12-198 .ssh]# chmod 600 authorized_keys
[root@ip-172-31-12-198 .ssh]# ls -l authorized_keys
-rw-------. 1 saish saish 92 Oct 20 06:35 authorized_keys
[root@ip-172-31-12-198 .ssh]#
```

Figure 18: The file permissions were changed to `600`, allowing only the owner to read and write the file.

5.9 Step 8: Exit and test SSH login as the new user

The root shell was exited, followed by the original `ec2-user` SSH session. From Windows, a new connection was attempted using the newly generated private key and the Linux username `saish`.

```
ssh -i "C:\Users\<WindowsUser>\.ssh\id_ed25519" saish@<EC2-Public-IP>
```

On the first connection to an address, SSH may ask whether to trust the server's host key. The host key identifies the server and is different from the user's login key. Verify the fingerprint through a trusted method before accepting it, especially in production environments.

After successful authentication, the following commands were run:

```
whoami
pwd
```

The output showed `saish` and `/home/saish`, confirming that the session was logged in as the new Linux user and opened in that user's home directory.

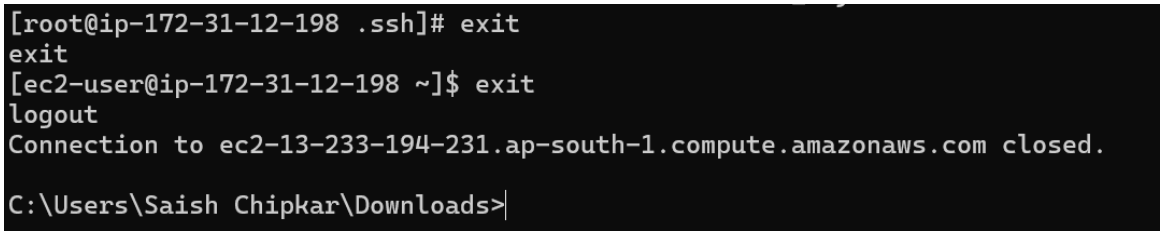


Figure 19: The first connection to the instance's IP address prompted for host-key confirmation.

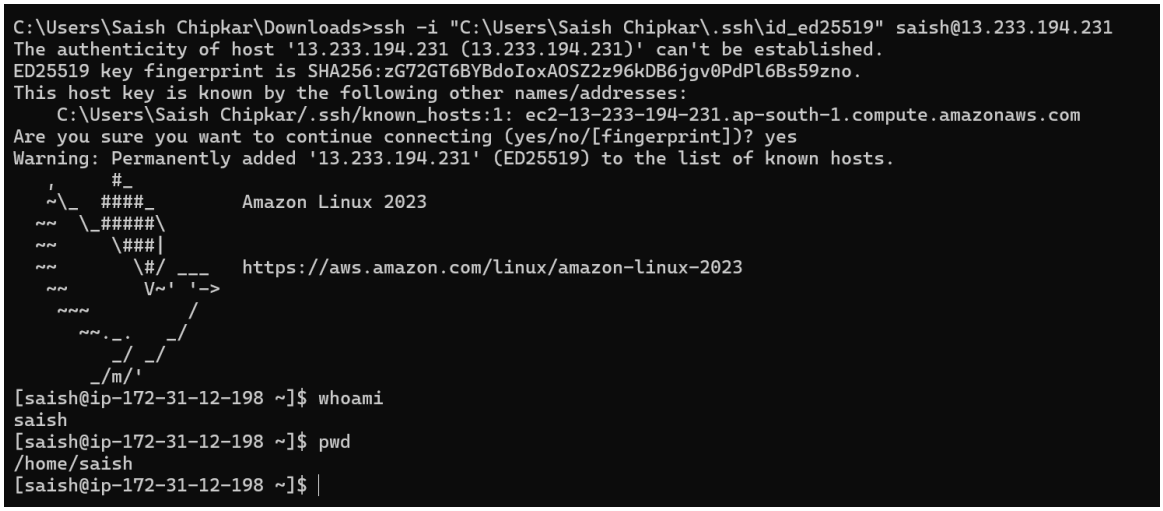


Figure 20: Successful SSH login as `saish`; `whoami` and `pwd` verify the account and home directory.

6 Results and Observations

1. An EC2 Linux instance was accessed initially using the original AWS PEM key and the `ec2-user` account.
2. A new Linux user, `saish`, was created.

3. A separate Ed25519 key pair was generated on Windows.
4. The matching public key was added to `/home/saish/.ssh/authorized_keys`.
5. The authorization file ownership was changed to `saish:saish` and its permissions were set to `600`.
6. A new SSH connection was made using the new private key and the username `saish`.
7. The `whoami` and `pwd` commands verified that the resulting session was running as `saish` in `/home/saish`.

7 Troubleshooting Notes

If the connection fails, check the following:

- **Wrong username:** Use the Linux account for which the public key was installed.
- **Wrong private key:** The private key must match the public key in `authorized_keys`.
- **Wrong path:** Ensure the `-i` argument points to the intended private-key file.
- **Permissions/ownership:** Confirm that `/home/saish`, `/home/saish/.ssh`, and `authorized_keys` have appropriate ownership and permissions. A common setup is `700` for `.ssh` and `600` for `authorized_keys`.
- **Network access:** The EC2 security group must allow inbound SSH (TCP port 22) from the client's public IP, and the instance must be reachable.
- **Server configuration:** Public-key authentication must be enabled, and the configured authorized-keys path must be correct.
- **Host-key prompt:** This verifies the server identity; it is separate from user authentication. Verify the fingerprint before accepting it.
- **Key overwritten:** If `ssh-keygen` overwrote an existing key, make sure the public key installed on the server corresponds to the private key currently being used.

8 Security Best Practices

- Never share or publish a private key, including in screenshots, reports, chat messages, or repositories.
- If a private key has been exposed, treat it as compromised: create a replacement key pair, install the replacement public key, test access, and remove the old public key from every account that trusts it.
- Use a separate key pair for each person or operational identity where practical. This makes access revocation and auditing easier.
- Keep the `.ssh` directory and `authorized_keys` file restricted to the account owner.
- Use a passphrase to protect private keys at rest where practical.

- Do not confuse the EC2 key pair used for initial login with the Linux user account. The username selects the account; the server checks which public keys that account accepts.
- Avoid using a root shell for routine work. Use `sudo` only for administrative operations.

9 Conclusion

This experiment demonstrated how SSH public-key authentication can be configured for a Linux user on an AWS EC2 instance. The private key remained on the Windows client, while the corresponding public key was installed in the target user's `authorized_keys` file. Ownership and permissions were configured, and the final login was verified using `whoami` and `pwd`. The key lesson is that the private key proves possession, while the server's per-user authorization configuration determines which Linux account can accept it.